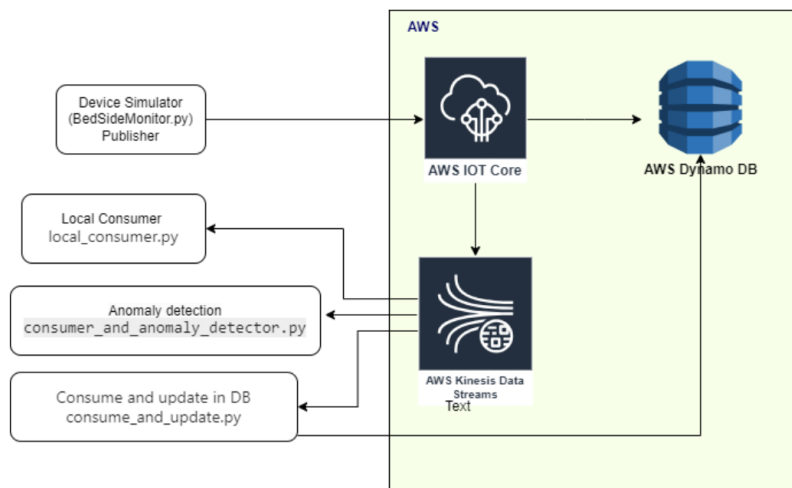


IoT-Core-Stream Processing-on Step-by-Step Guide

Overview

Requirement

Note: - You can do all of these in your AWS Free tier account.



In this hands-on, we are going to perform following tasks -

- Use the bedside monitor (from previous hands on) to push data in AWS IoT Core
- Create Data Stream in AWS Kinesis
- Create AWS IoT Core Rule to connect AWS IoT core with AWS Kinesis Data Stream and push incoming data from device into Kinesis Data Stream
- Writing a local python consumer to –
 - get data from data stream and display on console (local_consumer.py)
 - Creating rules to detect anomaly in the data coming from Kinesis data stream (consumer_and_anomaly_detector.py)
 - Storing anomalies in DynamoDB (consume_and_update.py)

Note : - All python source code is provide along with this hands on.

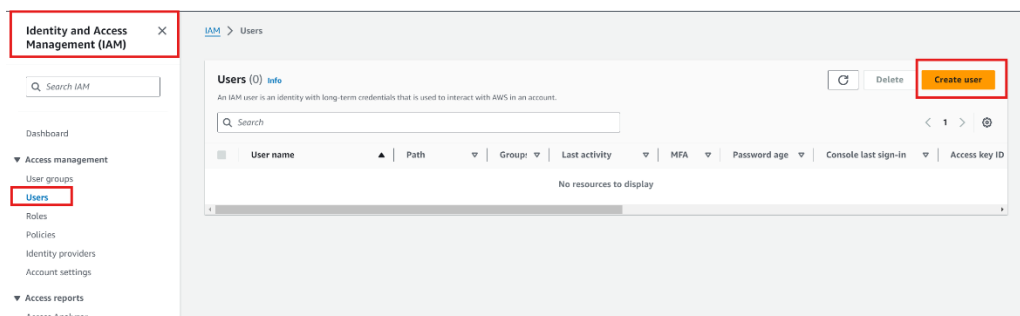
Connecting to AWS

- While connecting to AWS IoT Core we used **AWSIoTPythonSDK** and **awsiodtsdk** libraries and used the following parameters
 - endpoint (which we got from IOT Core -Setting) - This was like hostname.
 - certificates (root certificate and device certificate)
 - private key
 - topic name
- Now while connecting to other AWS Services like Kinesis, Dynamo DB, S3, etc. we will use the **boto3** library. This will require to set the following parameters which are stored in the configuration file. Steps to set this configuration is given below.
 - AWS region name
 - access & secret access keys of users

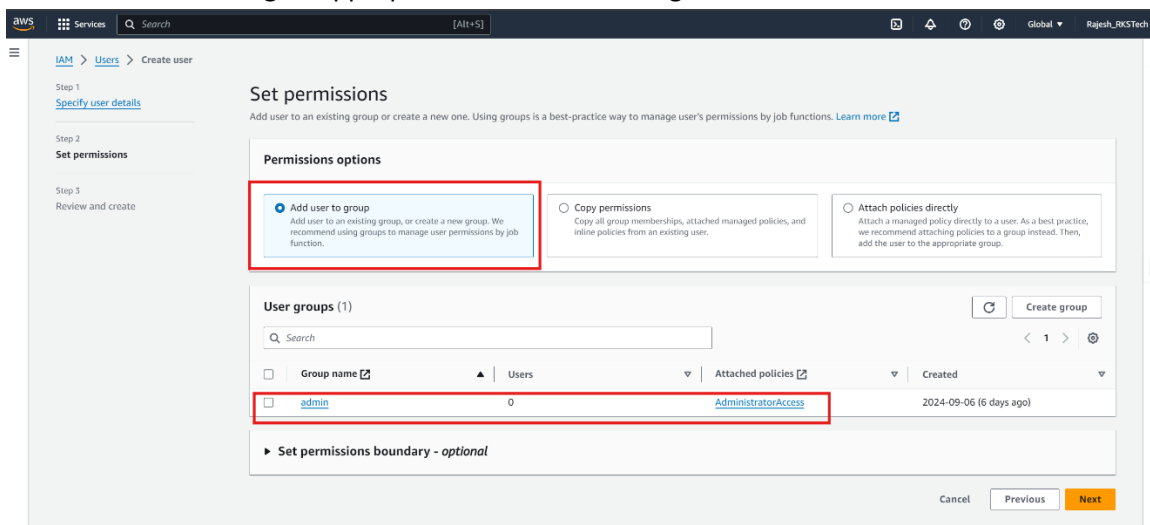
AWS configuration setup using command line

Get an AWS user

- If you do not have a AWS user, create a user name. Go to IAM and create user



- Give a username and give appropriate access. We have given admin access



- After the user is created go into user and create access keys.
This needs to be stored in the config files

Search IAM

Dashboard

Access management

- User groups
- Users**
- Roles
- Policies
- Identity providers
- Account settings

Access reports

- Access Analyzer
- External access
- Unused access
- Analyzer settings
- Credential report
- Organization activity
- Service control policies

Related consoles

- IAM Identity Center
- AWS Organizations

user1 Info

Delete

Summary

ARN
arn:aws:iam::343218226948:user/user1

Console access
Disabled

Created
September 13, 2024, 10:31 (UTC+05:30)

Last console sign-in
-

Access key 1
[Create access key](#)

Permissions Groups (1) Tags Security credentials Access Advisor

Permissions policies (1)

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type

Search

All types

Policy name Type Attached via

AdministratorAccess AWS managed - job function Group admin

Permissions boundaries (not set)

Step 2 - optional
Set description tag

Step 3
Retrieve access keys

Use case

☒ Command Line Interface (CLI)
You plan to use this access key to enable the AWS CLI to access your AWS account.

☐ Local code
You plan to use this access key to enable application code in a local development environment to access your AWS account.

☐ Application running on an AWS compute service
You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.

☐ Third-party service
You plan to use this access key to enable access for a third-party application or service that monitors or manages your AWS resources.

☐ Application running outside AWS
You plan to use this access key to authenticate workloads running in your data center or other infrastructure outside of AWS that needs to access your AWS resources.

☐ Other
Your use case is not listed here.

Alternatives recommended

- Use [AWS CloudShell](#), a browser-based CLI, to run commands. [Learn more](#)
- Use the [AWS CLI V2](#) and enable authentication through a user in IAM Identity Center. [Learn more](#)

Confirmation

☒ I understand the above recommendation and want to proceed to create an access key.

Cancel Next

IAM > Users > user1 > Create access key

Step 1
[Access key best practices & alternatives](#)

Step 2 - optional
[Set description tag](#)

Step 3
Retrieve access keys

Retrieve access keys Info

Access key

If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key

AKIAU72LGQ4CHQ7CSWN

Secret access key

U74nFdk4lgiaQ\$wM3z6+GtHPRE0WbeGMVpRGcNA [Hide](#)

Access key best practices

- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

For more details about managing access keys, see the [best practices for managing AWS access keys](#).

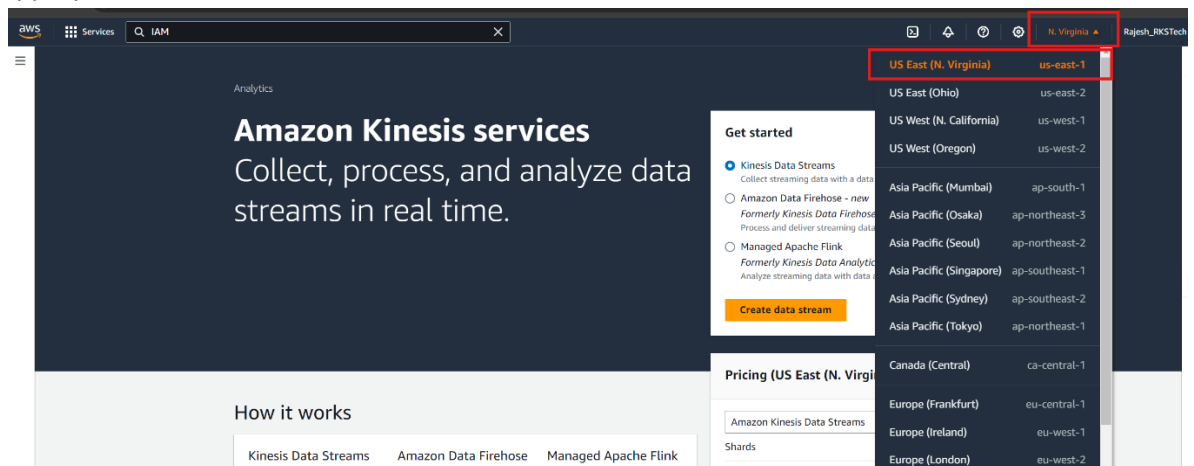
Download .csv file Done

AWS Command Line Interface

1. **Download and Install:** Go to the below link and download and install AWS CLI software for your OS. <https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>
2. ****Configure AWS CLI using aws configure.** Use AWS Access Key ID and AWS secret access key for setting up. Give the region name The key and Secret Access key will be from user id which was created in earlier step

```
c:\AWS>aws configure
AWS Access Key ID [*****ISD6]: AKIAU72LGQ4CLGNFS7VI
AWS Secret Access Key [*****IS+R]: 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
Default region name [us-east-1]: us-east-1
```

3. AWS Region you can get from console. Depending upon in which region you are working choose the appropriate one.



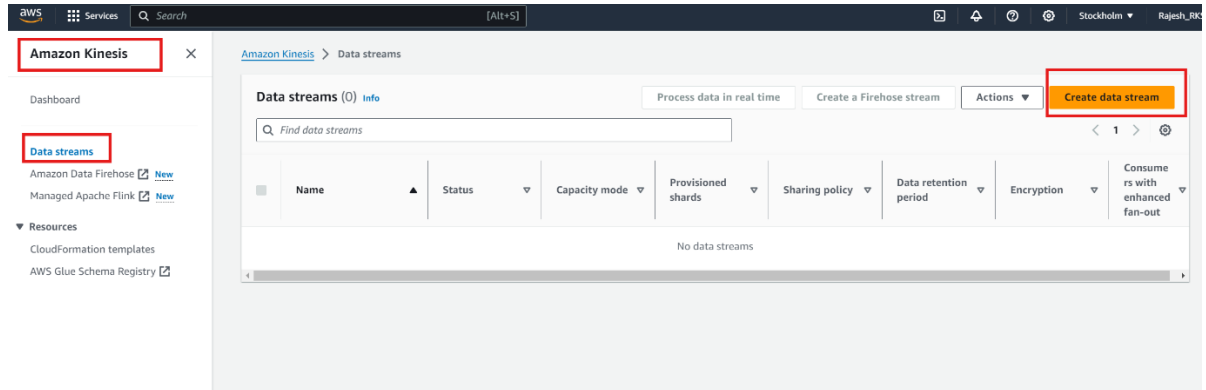
4. Refer to this AWS documentation for more details
- <https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-quickstart.html>



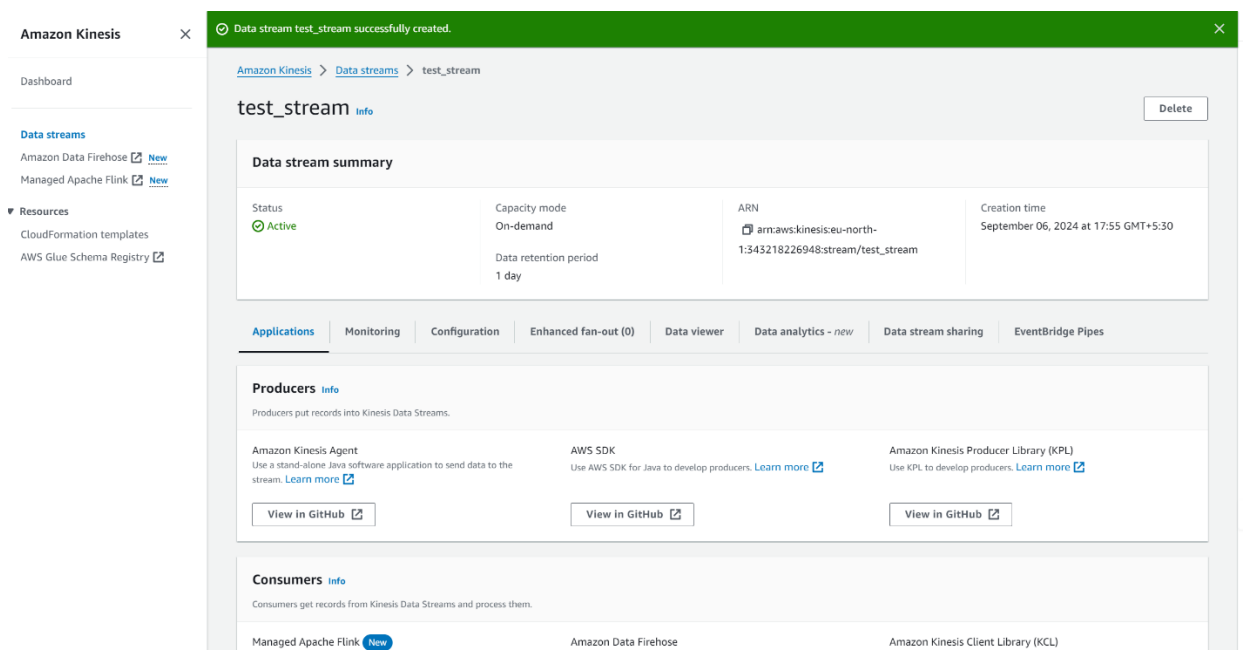
AWS Kinesis: Create Data Stream

Amazon Kinesis makes it easy to collect, process, and analyze data streams in real time, so you can get timely insights and react quickly to new information.

1. In AWS Console --> Open AWS Kinesis and Create a data stream



2. Enter a Data stream name. After you click Create data stream the below page will appear



AWS IoT Core: Create a Rule for Kinesis stream

Before we create a rule to push IoT data into a lambda function, let's create the role that should have permission to perform this operation.

1. Goto IAM and click on Create role similar to the way we have created in previous hands-on. Chosse 'IoT' use case.
2. Now we will create a rule and link the AWS Kinesis stream with AWS IoT Core
3. Go to AWS IoT Core page >> click on Act(or Messaging Routes) >> click on Rules >> click Create button . similar to the way we have created in previous hands-on.



4. In Create a rule page specify the Name of your rule and specify the topic ex- iot/test1000
5. In Rule Action click on Add action button. Select Kinesis Data Stream and select the Stream name from the stream which we have already created. Add deviceid in Partition key. Select the role we just created

Attach rule actions Info

An action routes data to a specific AWS service.

SQL statement Back

SELECT * FROM iot/test

Rule actions

Select one or more actions to happen when the above rule is matched by an inbound message. Actions define additional activities that occur when messages arrive, like storing them in a database, invoking cloud functions, or sending notifications. You can add up to 10 actions.

Action 1

Kinesis Data Stream A message sent to a VPC will be metered with an additional action. Remove

Stream name Info

test_stream View

Create Kinesis Stream

Partition key

The partition key is used to group data by shard in a stream. Try a generator function like \$(newuuid())

deviceid

IAM role

Choose a role to grant AWS IoT access to your endpoint.

RKSRole View Create new role

AWS IoT will automatically create a policy with a prefix of "aws-iot-rule" under your IAM role selected.

Add rule action

6. Once you are done you can click Create rule button. Rule will be created

AWS IoT ×

Successfully created rule BSM_Rule. View rule ×

Rules (1) Info

Rules allow your things to interact with other services. Rules are analyzed and perform specific actions based on messages published by your devices.

Find rules

☐	Name	Status	Rule topic	Created date
☐	BSM_Rule	Active	\$aws/null	September 06, 2024, 18:08:50 (UTC+05:30)

Create local consumer for Kinesis stream and detect anomalies

- We have linked AWS IoT core with Kinesis stream.
- Now, we will run our BedSideMonitor (device simulator) to push data in AWS IoT Core from where data will be further pushed into AWS Kinesis data stream using AWS IoT Rule which we created in above step.

1. Run BedSideMonitor.py program

Note – Ensure to change the Thing name in BedSideMonitor.py file with the Thing name you created in AWS IoT Core. (Statement - `message['deviceid'] = 'BSM_G101'`)

Command

```
python BedSideMonitor.py -e endpoint -r Root_certificate -c Thing_Certificate -k Thing_private_key -id clientid -t topic_name
```

Sample command

```
python BedSideMonitor.py -e a2yi6n1teb5l2d-ats.iot.us-east-1.amazonaws.com -r root-CA.cer -c 5a4e215a7c-certificate.pem.crt -k 5a4e215a7c-private.pem.key -id client001 -t iot/bsm
```

2. As the BedSideMonitor is running, now lets run our local consumer and get the data.

Run `local_consumer.py`

Note: Ensure to update the StreamName variable value in `local_consumer.py` file with the name of Data stream you created in AWS Kinesis.

```
1  Dimport boto3
2  import time
3  import json
4
5
6  client = boto3.client('kinesis')
7  shardIterator = client.get_shard_iterator(
8      StreamName='BSM_Data_Stream1',
9      ShardId='shardId-000000000003',
10     ShardIteratorType='LATEST',
11 )['ShardIterator']
12
13
14 while True:
15     response = client.get_records(
16         ShardIterator=shardIterator
17     )
18     shardIterator = response['NextShardIterator']
19     if len(response['Records']) > 0:
20         for item in response['Records']:
21             readings = json.loads(item["Data"])
22             print(readings)
23
24     time.sleep(0.2)
25
```

```
c:\AWS>python local_consumer.py
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:45:35.216442', 'datatype': 'HeartRate', 'value': 84}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:45:35.909829', 'datatype': 'HeartRate', 'value': 78}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:45:36.917444', 'datatype': 'HeartRate', 'value': 60}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:45:37.911606', 'datatype': 'HeartRate', 'value': 90}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:45:38.918702', 'datatype': 'HeartRate', 'value': 85}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:45:39.905568', 'datatype': 'HeartRate', 'value': 86}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:45:40.904381', 'datatype': 'HeartRate', 'value': 80}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:45:41.904466', 'datatype': 'HeartRate', 'value': 85}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:45:42.917043', 'datatype': 'HeartRate', 'value': 79}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:45:43.915335', 'datatype': 'HeartRate', 'value': 75}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:45:45.196121', 'datatype': 'SP02', 'value': 86}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:45:46.165483', 'datatype': 'HeartRate', 'value': 74}
```

- Now Run the program to detect anomalies in the data. Run consumer_and_anomaly_detector.py

NOTE : Make sure you update the StreamName variable value

in consumer_and_anomaly_detector.py file with the name of Data stream you created in AWS Kinesis.

```
c:\AWS>python consumer_and_anomaly_detector.py
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:49:11.361696', 'datatype': 'HeartRate', 'value': 65}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:49:12.363167', 'datatype': 'HeartRate', 'value': 99}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:49:13.361689', 'datatype': 'Temperature', 'value': 98.5}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:49:13.639684', 'datatype': 'HeartRate', 'value': 93}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:49:14.361351', 'datatype': 'HeartRate', 'value': 78}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:49:15.358384', 'datatype': 'HeartRate', 'value': 66}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:49:16.364410', 'datatype': 'HeartRate', 'value': 84}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:49:17.354955', 'datatype': 'HeartRate', 'value': 100}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:49:18.630458', 'datatype': 'HeartRate', 'value': 98}
```

Create a table in DynamoDB

As we have listed out anomalies in the data. We will have to store it.

We will create a table to store anomalies in DynamoDB

- In AWS go to DynamoDB >> Create table. follow the steps mentined in previous excercise

Store anomalies in DynamoDB table

As we have created a table in DynamoDB to store anomalies. Let's store the anomalies from BedSideMonitor in the table.

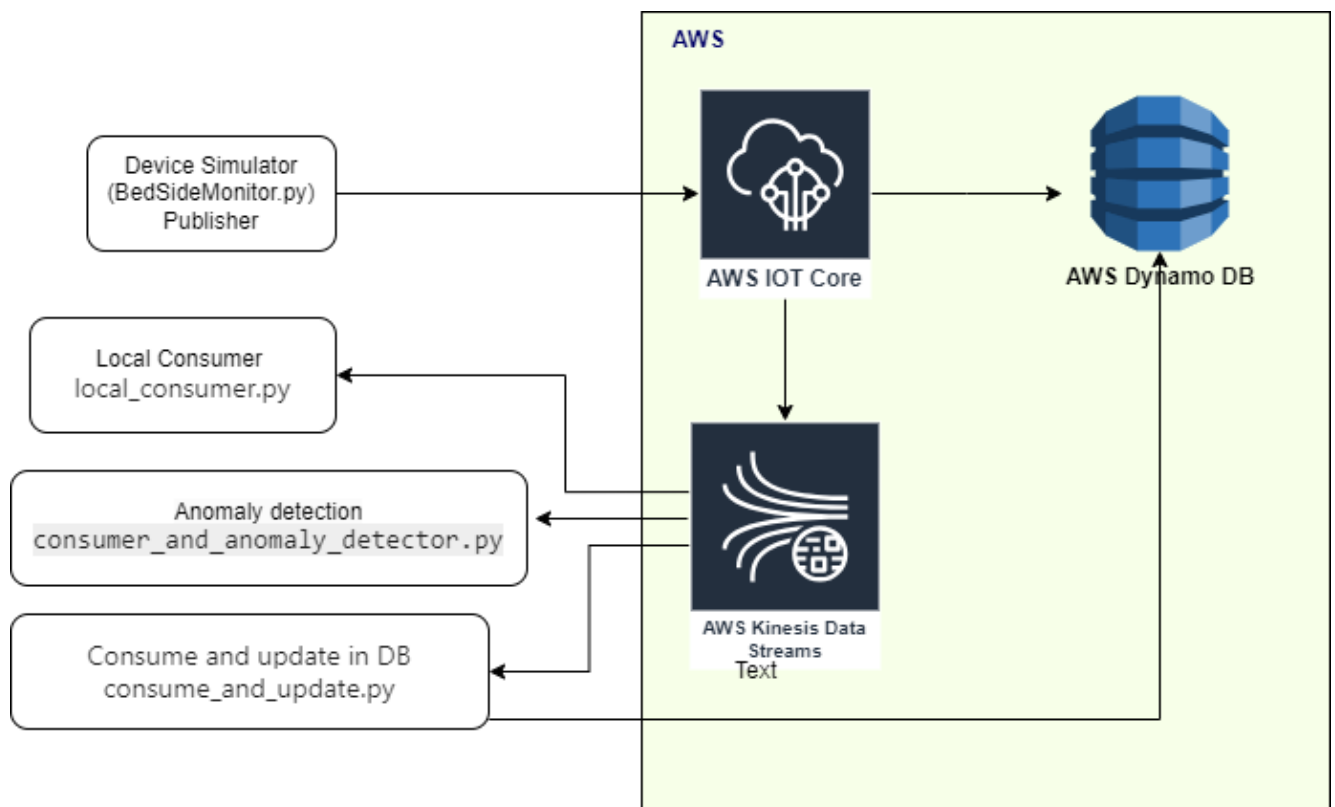
- Make sure BedSideMonnitor is running.
- Run the program to update anomalies in the table. Run consume_and_update.py

NOTE 1 : Make sure you update the StreamName variable value in consume_and_update.py file

with the name of Data stream you created in AWS Kinesis. NOTE 2 : Make sure you update the table name in the consume_and_update.py file with the table you created in DynamoDB.


```
c:\AWS>python consume_and_update.py
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:53:50.365971', 'datatype': 'HeartRate', 'value': 90}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:53:51.353906', 'datatype': 'HeartRate', 'value': 78}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:53:52.351885', 'datatype': 'HeartRate', 'value': 68}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:53:53.379629', 'datatype': 'HeartRate', 'value': 78}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:53:54.351854', 'datatype': 'HeartRate', 'value': 63}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:53:55.369644', 'datatype': 'HeartRate', 'value': 60}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:53:56.384897', 'datatype': 'HeartRate', 'value': 75}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:53:57.351691', 'datatype': 'HeartRate', 'value': 85}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:53:58.633932', 'datatype': 'SP02', 'value': 91}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:53:58.917184', 'datatype': 'HeartRate', 'value': 81}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:53:59.351459', 'datatype': 'HeartRate', 'value': 85}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:54:00.357943', 'datatype': 'HeartRate', 'value': 85}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:54:01.352913', 'datatype': 'HeartRate', 'value': 61}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:54:02.353495', 'datatype': 'HeartRate', 'value': 102}
Anomaly detected, entry added in DynamoDB Table
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:54:04.352096', 'datatype': 'HeartRate', 'value': 76}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:54:05.355619', 'datatype': 'HeartRate', 'value': 76}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:54:06.355382', 'datatype': 'HeartRate', 'value': 66}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:54:07.353400', 'datatype': 'HeartRate', 'value': 81}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:54:08.351448', 'datatype': 'SP02', 'value': 88}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:54:08.706039', 'datatype': 'HeartRate', 'value': 90}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:54:09.356019', 'datatype': 'HeartRate', 'value': 93}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:54:10.352352', 'datatype': 'HeartRate', 'value': 87}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:54:11.351949', 'datatype': 'HeartRate', 'value': 78}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:54:12.359315', 'datatype': 'HeartRate', 'value': 98}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:54:13.642085', 'datatype': 'HeartRate', 'value': 90}
{'deviceid': 'BSM_G101', 'timestamp': '2024-09-13 10:54:14.351979', 'datatype': 'HeartRate', 'value': 97}
```

Below is what we have created in this exercise



This collected data can now be used periodically for batch processing and analytics using a variety of tools.

Generally, your consumers will also be running in the cloud in an EC2 instance or a lambda function. We'll look at one such setup in the cloud module using Lambda as the consumer reading from Kinesis data analytics stream.



Important

- Once you have completed the hands-on, please ensure to delete all resources created on AWS to avoid any surprise charges.

